

Directorio de Médicos Especialistas — Documentación Técnica

Cliente: Ministerio de Educación de Guatemala · **Curso:** CC3106 Responsable AI, UVG **Equipo:** 4 personas · **Alcance:** Ciudad de Guatemala, zonas 1–18 **Repositorio:** github.com/nel-eleven11/REAL-Project1 · **Historial:** commits/main **Demo en vivo:** rai-proyecto1-502801.web.app **Fecha de esta versión:** 12 de agosto de 2026

1. Qué es este sistema

Directorio de médicos especialistas construido sobre Google Places API, con API paginada protegida por IP whitelist y UI de consulta. Además de recolectar, mide y publica sus propios sesgos (§8), implementa los límites de los Términos de Servicio de Google en código, no solo en prosa (§7), y expone un mecanismo de corrección y remoción para quienes aparecen sin haberlo consentido (§11). La IP whitelist se implementa como se pide, con sus límites documentados con honestidad (§6).

2. Arquitectura

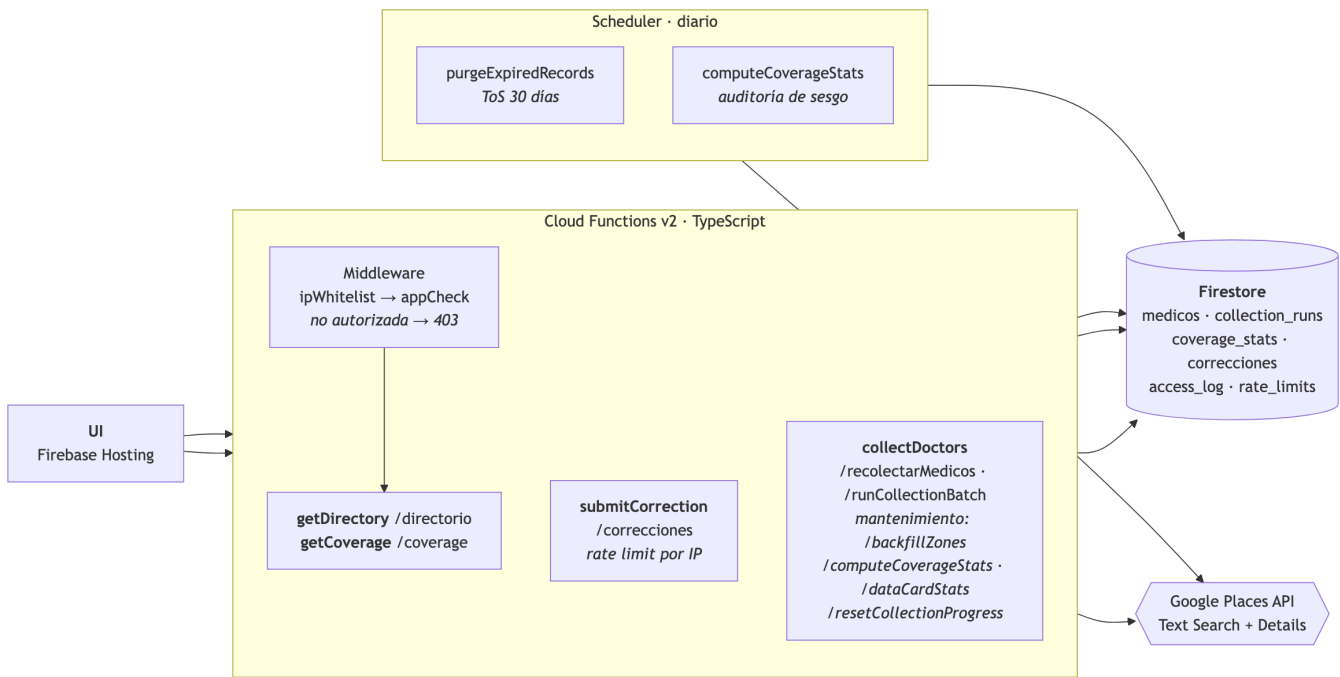


Figure 1: Diagrama de arquitectura: UI en Firebase Hosting llama al middleware de IP whitelist y App Check, que enruta a `getDirectory/getCoverage` y a `submitCorrection`, todos escribiendo a Firestore. `collectDoctors` llama a Google Places API y escribe a Firestore. Dos funciones programadas, `purgeExpiredRecords` y `computeCoverageStats`, corren a diario contra Firestore y Places.

Stack: TypeScript · Firebase Functions v2 · Firestore · Google Places API · Firebase Hosting. 90 % del desarrollo corre contra el emulador, sin costo. Los 768 registros que citan las cifras de este documento se recolectaron contra producción: auditar el sesgo de una fuente exige datos reales de esa fuente.

Cloud Functions

Función	Ruta / disparador	Responsabilidad
collectDoctors	GET /recolectarMedicos	Búsqueda puntual: keyword + zona + especialidad, consulta Places, guarda en Firestore. Tope 20 resultados por invocación.

Función	Ruta / disparador	Responsabilidad
collectDoctors	GET /runCollectionBatch	Recorre la matriz de 576 keywords con cursor persistido, en lotes de 1 a 10.
getDirectory	GET /directorío	Paginado con filtros. Excluye suppressed y vencidos.
getCoverage	GET /coverage	Sirve coverage_stats precalculado para el heatmap.
submitCorrection	POST /correcciones	Corrección o remoción. Sin IP whitelist, tiene que ser público, protegido por rate limit.
purgeExpiredRecordsScheduled	Diario	Refresca o purga documentos vencidos. Cumplimiento ToS.
computeCoverageStatsScheduled	Diario	Recalcula la matriz zona x especialidad.

Cuatro rutas de mantenimiento más, bajo la misma IP whitelist y sin costo de Places API: /computeCoverageStats dispara el cálculo diario a mano, /dataCardStats agrega las cifras de la Data Card, /backfillZones recalcula zona sobre documentos ya guardados, y /resetCollectionProgress reinicia el cursor tras un cambio de orden en la matriz.

3. Modelo de datos

place_id es la clave del documento en medicos, el identificador estable de Google. Eso convierte la reinserción en un upsert idempotente: la deduplicación es propiedad del esquema.

```
medicos/{place_id}
  nombre, direccion, telefono|null, sitio_web|null, lat|null, lng|null
  especialidad          # efectiva, el campo por el que filtra /directorío
  especialidad_raw      # la especialidad con la que se buscó (trazabilidad)
  especialidad_normalizada # derivada del nombre por reglas | null
  confidence            # 0.9 si hubo match, 0 si no
  zona                 # efectiva, el campo por el que filtra /directorío
  zona_raw             # la zona con la que se buscó (trazabilidad)
  zona_normalizada     # extraída de la dirección por regex | null
  missing_fields: string[] # ausencias explícitas, nunca cadenas vacías
  fecha_recoleccion, expires_at, run_id, keyword_usado
  suppressed: boolean    # true tras una solicitud de remoción
```

Especialidad y zona siguen la misma forma de tres campos: el buscado nunca se sobrescribe, el derivado puede quedar null, y un tercero resuelve cuál gana. Places no expone ninguno como campo propio, así que ambos salen del único texto libre disponible, nombre y dirección.

Colecciones de apoyo: collection_runs guarda telemetría por corrida, coverage_stats la matriz precalculada, correcciones la cola de revisión, access_log cada 200, 403 y 429, rate_limits la ventana fija de 60 segundos, collection_progress/state el cursor compartido.

Normalización de especialidades

Google Places no tiene un campo de especialidad médica. Lo único disponible es el nombre del negocio como texto libre. Evaluamos dos formas de resolver eso y elegimos la primera:

- **Reglas determinísticas, implementada.** Un diccionario de raíces por especialidad escanea el nombre, sin distinguir acentos ni género o número. Si hay coincidencia, especialidad_normalizada se llena con confianza 0.9. Si no, queda null con confianza 0. Nunca se inventa una especialidad.
- **LLM, descartada.** Exigiría un set de evaluación etiquetado a mano, al menos 100 registros, y presupuesto de inferencia que no estaba contemplado. El criterio de descarte no fue solo el costo: una especialidad alucinada puede derivar en daño al paciente, y ese error pesa más que la cobertura adicional que un modelo aportaría.

/directorío filtra por especialidad = especialidad_normalizada ?? especialidad_raw: lo que el lugar aparenta ser gana sobre lo que buscamos. Una “Clínica Pediátrica” encontrada con keyword de cardiología aparece bajo

pediatría. Cuando el nombre no revela nada, la especialidad buscada es la única señal disponible, y preferimos una señal débil pero declarada a esconder el registro.

Normalización de zonas

La zona buscada tampoco es confiable por sí sola. Text Search puede devolver lugares fuera de la zona consultada, incluso fuera de Ciudad de Guatemala, porque hace match por nombre y tipo de negocio, no por ubicación. Buscar “clínica pediátrica zona 5 Guatemala” puede devolver una clínica en Escuintla, y guardarla bajo zona 5 habría contaminado la matriz de la auditoría (§8).

zona_normalizada se extrae de direccion por regex sobre el patrón zona N, con valores de 1 a 25. Si la dirección no la menciona, queda null y zona cae a zona_raw. Igual que con especialidad, no se asume: el fallback está declarado, no disfrazado de dato verificado.

Nota metodológica: coverage_stats agrupa por zona_raw, no por la zona efectiva. El porqué está en docs/auditoria-cobertura.md §3, decisión 4.

4. Recolección de datos

Patrón de keyword: "<sufijo> <especialidad> <zona> Guatemala", con 4 sufijos, médico, doctor, clínica y consultorio, porque Google Maps no tiene nomenclatura consistente para consultorios médicos.

Matriz: 8 especialidades × 4 sufijos × 18 zonas, 576 combinaciones.

Orden ancho-primero. La matriz se recorre sufijo, zona, especialidad, no zona, especialidad, sufijo. Con cuota diaria limitada la recolección toma varios días; agotar una zona antes de tocar las demás dejaría el heatmap parcial marcando casi todo como “nunca buscado” cuando ya se buscó, solo que concentrado. Con este orden, los primeros 144 combos cubren cada celda de la grilla una vez antes de repetir ninguna.

Cobertura balanceada obligatoria. Mismo número de búsquedas en cada zona sin importar cuántos resultados devuelva. Concentrar el esfuerzo donde ya hay resultados mediría nuestro propio criterio de búsqueda, no la realidad de la fuente.

Cada invocación consume 1 llamada de Text Search más hasta 20 de Place Details, porque teléfono y sitio web no vienen en Text Search, y escribe un collection_runs con keyword, zona, resultados nuevos contra duplicados y costo estimado. runCollectionBatch avanza un cursor compartido en Firestore para que cualquier integrante continúe donde otro se quedó.

5. API

GET /directorio, paginado, con filtros opcionales.

Parámetro	Tipo	Default	Límite
page	entero ≥ 1	1	—
pageSize	entero ≥ 1	20	50, se recorta, no se rechaza
especialidad	string	—	filtra por especialidad efectiva
zona	string	—	filtra por zona efectiva; se buscan zona 1 ... zona 18

El filtro acepta zonas fuera del rango buscado: la zona efectiva sale de la dirección real (§3) y puede llegar hasta zona 25. Esos registros no se recortan, son resultados legítimos de Places fuera del alcance de búsqueda, pero la grilla de la auditoría sigue siendo la de las zonas 1 a 18.

Respuesta: { results, page, pageSize, hasMore }. Entradas inválidas caen al default en vez de devolver 400: un pageSize=abc no debe romper la UI.

suppressed y expires_at se filtran en memoria, no en la consulta de Firestore, para no exigir un índice compuesto por cada combinación de filtros. Eso podría dejar una página a medio llenar, así que la consulta sobre-lee con duplicación progresiva del límite, hasta 5 rondas, antes de leer todo.

6. Seguridad y modelo de amenazas

La IP whitelist se implementa como middleware, primer paso de la request: si la IP no está autorizada devuelve 403 sin ejecutar ninguna lógica adicional. Toda decisión, 200, 403 o 429, queda en `access_log`.

La IP se deriva del proxy de confianza de Cloud Functions. No se confía en X-Forwarded-For crudo, que el cliente puede falsificar; hay un test automatizado que envía el header falso y exige 403.

Por qué la whitelist no basta:

Debilidad	Impacto	Mitigación aplicada
X-Forwarded-For falsificable si se lee mal	Bypass total	IP del proxy de confianza + test de spoofing
Es ubicación de red, no autenticación	Cualquiera dentro de la IP entra	Rate limit por IP + auditoría
IPs dinámicas o red móvil	Falsos negativos, acceso roto	Limitación operativa documentada
No frena abuso desde IP autorizada	Extracción masiva del dataset	Rate limit + tope de 50 por página

Defensa en profundidad: Firebase App Check en la UI, rate limit por IP en `/correcciones`, `access_log` auditable, y API key en variables de entorno, nunca en el código, restringida a Places API en la consola de GCP y sin atarla a una IP, porque las llamadas salen desde la infraestructura de Google, no desde una IP fija nuestra.

Cierre del hueco de configuración. Que `APP_CHECK_ENFORCE` existiera en el código no garantizaba nada: si nadie la ponía en `true` al desplegar, quedaba inerte sin más señal que un log que nadie revisa. `firebase.json` corre un script de predeploy que aborta el deploy si el proyecto destino no es de emulador y falta `APP_CHECK_ENFORCE=true`. Declarar la variable dejó de ser suficiente, el pipeline lo hace cumplir.

7. Cumplimiento de los Términos de Servicio

Los ToS de Google Maps Platform permiten almacenar `place_id` indefinidamente, pero el resto del contenido solo puede cachearse hasta 30 días calendario consecutivos. Guardar nombres y teléfonos para siempre por ser proyecto académico sería incumplimiento.

Cada documento lleva `expires_at = fecha_recoleccion + 30 días`, y la purga diaria hace:

- **Refresco:** re-consulta por `place_id`, reemplaza el contenido y renueva la ventana.
- **Purga:** elimina el contenido y conserva `place_id` más un `purge_reason`: `suppressed`, `no_api_key`, `not_found_in_places` o `refresh_error`. “Google confirmó que cerró” y “no teníamos la key configurada” producirían documentos idénticos si no se registrara el motivo, y el segundo caso es un problema de configuración disfrazado de dato.

Un registro `suppressed` nunca se refresca, se purga directo: refrescar a alguien que pidió su remoción desharía la remoción. La purga borra también los campos derivados, `especialidad`, `confidence`, `zona_raw` y `zona_normalizada`, porque haberlos calculado nosotros no los saca del alcance de los ToS si su insumo vino de Places.

8. Auditoría de cobertura y sesgo

Con los mismos datos ya recolectados, costo de API adicional cercano a cero, se calcula por celda zona x especialidad: búsquedas ejecutadas, resultados únicos, porcentaje con teléfono y con sitio web.

Recolección del 12 de agosto de 2026: 122 de 144 celdas con al menos un resultado, 22 buscadas sin devolver ninguno, y sobre las celdas con datos 87.16 % con teléfono frente a 43.27 % con sitio web, una brecha que ya es indicador de digitalización parcial. El heatmap alterna entre ambas métricas, porque una celda puede ser densa en resultados y pobre en contacto a la vez.

El detalle metodológico y el hallazgo completo están en `docs/auditoria-cobertura.md`. `searches_run` se cuenta desde `collection_runs`, no desde `medicos`: si se contara desde los resultados, una zona buscada diez veces sin encontrar nada se vería igual que una nunca buscada, ambas en cero, y esa distinción es justo lo que la auditoría necesita capturar.

9. Costos

Text Search, unos \$0.032 por llamada, y Place Details, unos \$0.017, tienen tarifas distintas. Sumarlas a una sola tarifa subestimaba el gasto, así que `collection_runs` las contabiliza por separado.

Métrica	Valor
Corridas / llamadas a la API	349 · 5,821
Gasto teórico a tarifa de lista	\$104.19 USD
Cargo real facturado	\$0.00 USD
Registros únicos	768
Costo teórico por médico único	≈ \$0.136 USD
Resultados nuevos / duplicados	768 / 4,704
Tasa de duplicados	85.96 %

Una tasa de duplicados de casi 86 % no es un error: con 4 sufijos por celda es esperable que el segundo, tercero y cuarto vuelvan a encontrar el mismo lugar que el primero, costo de probar varios sufijos para no perder resultados que solo aparecen bajo uno de ellos.

Corrección sobre el crédito de \$200. El enunciado asume un crédito mensual fijo de \$200 en Places API, descontinuado el 1 de marzo de 2025 y reemplazado por 10,000 llamadas gratis al mes por producto en el nivel Essentials. Las 5,821 llamadas caen completas en esa cuota, así que el cargo real fue \$0.00, verificado en la consola de facturación de GCP. Los \$104.19 se reportan igual porque miden disciplina de gasto por llamada, dato que importa aunque el saldo final sea cero.

Controles preventivos: alertas de billing al 50 % y 90 % de un presupuesto propio, cuota máxima diaria de llamadas, y desarrollo en emulador. Cada integrante usa su propio proyecto GCP y responde por su propio gasto.

10. Pruebas y CI

39 pruebas de integración corren contra el emulador de Firebase en GitHub Actions, costo de API cero. Cubren IP no autorizada, X-Forwarded-For falsificado, tope de pageSize, purga y purge_reason, un suppressed que no reaparece, deduplicación por place_id, extracción de zona, el cursor de recolección y un recorrido end-to-end completo.

Correr la suite contra producción consumiría cuota real; ese paso es checklist manual previo a la demo, documentado en el README.

11. Postura ética

- **ToS:** la ventana de 30 días está implementada en código, no declarada en prosa.
- **No fabricación:** no se agregan ni infieren datos ausentes. Los campos faltantes se guardan como null con missing_fields, y la UI muestra “No reportado en la fuente”, nunca un espacio en blanco que parezca dato. El campo sitio_web puede estar vacío o apuntar a una clínica en lugar de a un sitio propio; se documenta como viene.
- **Sesgo de fuente:** publicamos la auditoría de cobertura y su hallazgo, incluso cuando el hallazgo debilita la utilidad aparente del producto.
- **Uso previsto y prohibido:** el directorio es referencia informativa, no validación de credenciales médicas. Se declara en la UI y en la Data Card.
- **Frescura:** fecha_recoleccion acompaña todo dato mostrado o exportado, con su antigüedad en días.
- **Derechos de terceros:** los médicos listados no consintieron aparecer aquí. Que su información sea pública en Google Maps no equivale a consentimiento para redistribuirla. El mecanismo de corrección y remoción es operativo y visible en la UI, no escondido en un pie de página, y una remoción sobrevive a recolecciones futuras.
- **Alcance:** los datos no se redistribuyen como producto independiente. Es estrictamente académico; producción real exigiría un acuerdo comercial con Google.

12. Limitaciones conocidas

La limitación central es que la cobertura refleja presencia digital en Google Maps, no oferta médica: una celda vacía significa que Google no lo tiene, nunca que ahí no hay médicos. Le siguen que especialidad y zona son inferidas o asumidas pero nunca verificadas, que Text Search puede devolver lugares fuera del alcance geográfico consultado, que la whitelist protege ubicación de red y no identidad, y que el dato puede tener hasta 30 días de antigüedad. El inventario completo está en docs/data-card.md §5 y el detalle del sesgo en docs/auditoria-cobertura.md.